

ALFRED

A Benchmark for Interpreting Grounded Instructions for Everyday Tasks

Mohit Shridhar¹
Winson Han³

Jesse Thomason¹
Roozbeh Mottaghi^{1,3}

Daniel Gordon¹
Luke Zettlemoyer¹

Yonatan Bisk^{1,2,3}
Dieter Fox^{1,4}

AskForALFRED.com

Abstract

We present **ALFRED** (Action Learning From Realistic Environments and Directives), a benchmark for learning a mapping from natural language instructions and ego-centric vision to sequences of actions for household tasks. ALFRED includes long, compositional tasks with non-reversible state changes to shrink the gap between research benchmarks and real-world applications. ALFRED consists of expert demonstrations in interactive visual environments for 25k natural language directives. These directives contain both high-level goals like “Rinse off a mug and place it in the coffee maker.” and low-level language instructions like “Walk to the coffee maker on the right.” ALFRED tasks are more complex in terms of sequence length, action space, and language than existing vision-and-language task datasets. We show that a baseline model based on recent embodied vision-and-language tasks performs poorly on ALFRED, suggesting that there is significant room for developing innovative grounded visual language understanding models with this benchmark.

1. Introduction

A robot operating in human spaces must learn to connect natural language to the world. This *symbol grounding* [21] problem has largely focused on connecting language to static images. However, robots need to understand task-oriented language, for example “Rinse off a mug and place it in the coffee maker,” as illustrated in Figure 1.

Platforms for translating language to action have become increasingly popular, spawning new test-beds [3, 12, 14, 42]. These benchmarks include language-driven navigation and embodied question answering, which have seen dramatic improvements in modeling thanks to environments like Matterport 3D [3, 11], AI2-THOR [26], and AI Habitat [45]. However, these datasets ignore complexities arising from describing task-oriented behaviors with objects.

We introduce **ALFRED**, a new benchmark for con-

Goal: “Rinse off a mug and place it in the coffee maker”



Figure 1: ALFRED consists of 25k language directives corresponding to expert demonstrations of household tasks. We highlight several frames corresponding to portions of the accompanying language instruction. ALFRED involves interactions with objects, keeping track of state changes, and references to previous instructions.

necting human language to *actions*, *behaviors*, and *objects* in interactive visual environments. Planner-based expert demonstrations are accompanied by both high- and low-level human language instructions in 120 indoor scenes in AI2-THOR 2.0 [26]. These demonstrations involve partial observability, long action horizons, underspecified natural language, and irreversible actions.

ALFRED includes 25,743 English language directives describing 8,055 expert demonstrations averaging 50 steps each, resulting in 428,322 image-action pairs. Motivated by work in robotics on segmentation-based grasping [37], agents in ALFRED interact with objects visually, specifying a pixelwise interaction mask of the target object. This inference is more realistic than simple object class prediction, where localization is treated as a solved problem. Existing beam-search [17, 48, 53] and backtracking solutions [24, 29] are infeasible due to the larger action and state spaces, long horizon, and inability to undo certain actions.

¹Paul G. Allen School of Computer Sci. & Eng., Univ. of Washington

²Language Technologies Institute @ Carnegie Mellon University

³Allen Institute for AI ⁴NVIDIA

	— Language —		— Virtual Environment —			— Inference —		
	# Human Annotations	Granularity	Visual Quality	Movable Objects	State Changes	Vis. Obs.	Navigation	Interaction
TACoS [43]	17k+	High&Low	Photos	✗	✗	—	—	—
R2R [3]; Touchdown [14]	21k+; 9.3k+	Low	Photos	✗	✗	Ego	Graph	✗
EQA [15]	✗	High	Low	✗	✗	Ego	Discrete	✗
Matterport EQA [55]	✗	High	Photos	✗	✗	Ego	Discrete	✗
IQA [20]	✗	High	High	✗	✓	Ego	Discrete	Discrete
VirtualHome [42]	2.7k+	High&Low	High	✓	✓	3 rd Person	✗	Discrete
VSP [58]	✗	High	High	✓	✓	Ego	✗	Discrete
ALFRED 	25k+	High&Low	High	✓	✓	Ego	Discrete	Discrete + Mask

Table 1: **Dataset comparison.** ALFRED is the first interactive visual dataset to include high-level goal and low-level natural language instructions for object and environment interactions. TACoS [43] provides detailed high- and low-level text descriptions of cooking videos, but does not facilitate task execution. For navigation, ALFRED enables discretized, grid-based movement, while other datasets use topological graph navigation or avoid navigation altogether. ALFRED requires an agent to generate spatially located interaction masks for action commands. By contrast, other datasets only require choosing from a discrete set of available interactions and object classes or offer no interactive capability.

To establish baseline performance levels, we evaluate a sequence-to-sequence model akin to existing vision-and-language navigation tasks [28]. This model is not effective on the complex tasks in ALFRED, achieving less than 5% success rates. For analysis, we also evaluate individual sub-goals. While performance is better for isolated sub-goals, the model lacks the reasoning capacity for long-horizon and compositional task planning.

In summary, ALFRED facilitates learning models that translate from language to sequences of actions and interactions in a visually and physically realistic simulation environment. This benchmark captures many challenges present in real-world settings for translating human language to robot actions for accomplishing household tasks. Models that can overcome these challenges will begin to close the gap towards real-world, language-driven robotics.

2. Related Work

Table 1 summarizes the benefits of ALFRED relative to other visual action datasets with language annotations.

Vision & Language Navigation. In vision-and-language navigation tasks, either natural or templated language describes a route to a goal location through egocentric visual observations [3, 12, 13, 14, 31]. Since the proposal of R2R [3], researchers have dramatically improved the navigation performance of models [17, 24, 29, 53, 54] with techniques like progress monitoring [28], as well as introduced task variants with additional, on-route instructions [38, 39, 51]. Much of this research is limited to static environments. By contrast, ALFRED tasks include navigation, object interactions, and state changes.

Vision & Language Task Completion. There are sev-

eral existing benchmarks based on simple block worlds and fully observable scenes [9, 34]. ALFRED provides more difficult tasks in richer, visually complex scenes, and uses partially observable environments. The CHAI benchmark [33] evaluates agents performing household instructions, but uses a generic “interact” action. ALFRED has seven manipulation actions, such as pick up, turn on, and open, state changes like clean versus dirty, and variation in language and visual complexity.

Previous work in the original AI2-THOR environment investigated the task of visual semantic planning [19, 58]. Artificial language came from templates, and environment interaction was handled with discrete class predictions, for example selecting *apple* as the target object from predefined options. ALFRED features human language instructions, and object selections are carried out with class-agnostic, pixelwise interaction masks. In VirtualHome [42], programs are generated from video demonstration and natural language instructions, but inference does not involve ego-centric visual and action feedback or partial observability.

There is an extensive literature on language-based instruction following in the natural language processing community. There, research has focused on mapping instructions to actions [5, 13, 32, 36, 49], but these works do not involve visual, interactive environments.

Embodied Question Answering. Existing datasets for visual question answering in embodied environments use templated language or static scenes [15, 20, 55, 57]. In ALFRED, rather than answering a question, the agent must complete a task specified using natural language, which requires both navigation and interaction with objects.

Instruction Alignment. Language annotations of videos enable discovering visual correspondences between words

	Pick & Place	Stack & Place	Pick Two & Place	Clean & Place	Heat & Place	Cool & Place	Examine in Light
item(s)	Book	Fork (in) Cup	Spray Bottle	Dish Sponge	Potato Slice	Egg	Credit Card
receptacle	Desk	Counter Top	Toilet Tank	Cart	Counter Top	Side Table	Desk Lamp
scene #	Bedroom 14	Kitchen 10	Bathroom 2	Bathroom 1	Kitchen 8	Kitchen 21	Bedroom 24
expert demonstration							

	Annotation # 1	Annotation # 2	Annotation # 3
Goals	Put a clean sponge on a metal rack.	Place a clean sponge on the drying rack	Put a rinsed out sponge on the drying rack
Instructions	Go to the left and face the faucet side of the bath tub. Pick up left most green sponge from the bath tub. Turn around and go to the sink. Put the sponge in the sink. Turn on then turn off the water. Take the sponge from the sink. Go to the metal bar rack to the left. Put the sponge on the top rack to the left of the lotion bottle.	Turn around and walk over to the bathtub on the left. Grab the sponge out of the bathtub. Turn around and walk to the sink ahead. Rinse the sponge out in the sink. Move to the left a bit and face the drying rack in the corner of the room. Place the sponge on the drying rack.	Walk forwards a bit and turn left to face the bathtub. Grab a sponge out of the bathtub. Turn around and walk forwards to the sink. Rinse the sponge out in the sink and pick it up again. Turn left to walk a bit, then face the drying rack. Put the sponge on the drying rack.

Figure 2: **ALFRED annotations.** We introduce 7 different task types parameterized by 84 object classes in 120 scenes. An example of each task type is given above. For the **Clean & Place** demonstration, we also show the three crowdsourced language directives. Please see the supplemental material for example demonstrations and language for each task.

and concepts [1, 46, 43, 56, 59]. ALFRED requires performing tasks in an interactive setting as opposed to learning from recorded videos.

Robotics Instruction Following. Instruction following is a long-standing topic of interest in robotics [7, 10, 30, 35, 40, 41, 47, 52]. Lines of research consider different tasks such as cooking [10], table clearing [40], and mobile manipulation [30]. In general, they are limited to a few scenes [35], consider a small number of objects [30], or use the same environment for training and testing [7]. In contrast, ALFRED includes 120 scenes, many object classes with diverse appearances, and a test set of unseen environments.

3. The ALFRED Dataset

The ALFRED dataset comprises 25,743 language directives corresponding to 8,055 expert demonstration episodes. Each directive includes a high-level goal and a set of step-by-step instructions. Each expert demonstration can be deterministically replayed in the AI2-THOR 2.0 simulator.

3.1. Expert Demonstrations

Expert demonstrations are composed of an agent’s ego-centric visual observations of the environment and what action is taken at each timestep as well as ground-truth interaction masks. These demonstrations are generated by a planner [23] using metadata not available to the agent at inference time. Navigation actions move the agent or change its camera orientation, while manipulation actions include picking and placing objects, opening and closing cabinets and drawers, and turning appliances on and off. Interactions can involve multiple objects, such as using a knife to slice an apple, cleaning a mug in the sink, and heating a potato in the microwave. Manipulation actions are accompanied by a ground truth segmentation of the target object.

Figure 2 gives examples of the high-level agent tasks in ALFRED, like putting a cleaned object at a destination. These tasks are parameterized by the object of focus, the destination receptacle (e.g., *table top*), the scene in which to carry out the task, and in the case of **Stack & Place**, a base object (e.g., *plate*). ALFRED contains expert demonstrations of these seven tasks executed using combinations of 58 unique object classes and 26 receptacle object classes across 120 different indoor scenes. For object classes like *potato slice*, the agent must first pick up a *knife* and find a *potato* to create slices. All object classes contain multiple visual variations with different shapes, textures, and colors. For example, there are 30 unique variants of the *apple* class. Indoor scenes include different room types: 30 each of kitchens, bathrooms, bedrooms, and living rooms.

For 2,685 combinations of task parameters, we generate three expert demonstrations per parameter set, for a total of 8,055 unique demonstrations with an average of 50 action steps. The distributions of actions steps in ALFRED demonstrations versus related datasets is given in Figure 3. As an example, for task parameters {task: **Heat & Place**, object: *potato*, destination: *counter top*, scene: KITCHEN-8}, we generate three different expert demonstrations by starting the agent and objects in randomly chosen locations. Object start positions have some commonsense, class-specific constraints, for example a *fork* can start inside a *drawer*, but an *apple* cannot.

Contrasting navigation-only datasets where expert demonstrations can come from an A^* planner, our state space includes object positions and state changes. Thus, to generate expert demonstrations we encode the agent and object states, as well as high-level environment dynamics, into Planning Domain Definition Language (PDDL) rules [18]. We then define task-specific PDDL goal conditions, for example that a heated *potato* is resting on a *table top*. Note

	Train	Validation		Test	
		Seen	Unseen	Seen	Unseen
# Annotations	21,023	820	821	1,533	1,529
# Scenes	108	88	4	107	8

Table 2: **ALFRED Data Splits.** All expert demonstrations and associated language directives in the validation and test folds are distinct from those in the train fold. The validation and test sets are split into *seen* and *unseen* folds. Scenes in the *seen* folds of validation and test data are subsets of those in the train fold. Scenes in the *unseen* validation and test folds are distinct from the train folds and from each other.

that the planner encodes the environment as *fully observable* and has perfect knowledge about world dynamics. For training and testing agent models, however, the environment is *partially observable*: it is only viewed through the agent’s egocentric vision as actions are carried out.

We split these expert demonstrations into training, validation, and test folds (Table 2). Following work in vision-and-language navigation [3], we further split the validation and test into two conditions: *seen* and *unseen* environments. This split facilitates examining how well models generalize to entirely new spaces with novel object class variations.

3.2. Language Directives

For every expert demonstration, we collect open vocabulary, free-form language directives from at least three different annotators using Amazon Mechanical Turk (AMT), resulting in 25k total language directives. Language directives include a high-level *goal* together with low-level *instructions*, as shown in Figures 1 and 2. The distribution of language annotation token lengths in ALFRED versus related datasets is given in Figure 3.

AMT workers are told to write instructions to tell a “smart robot” how to accomplish what is shown in a video. We create a video of each expert demonstration and segment it such that each segment corresponds to an instruction. We consult the PDDL plan for the expert demonstration to identify task sub-goals, for example the many low-level steps to navigate to a knife, or the several steps to heat a potato slice in the microwave once standing in front of it. We visually highlight action sequences related to sub-goals via colored timeline bars below the video. In each HIT (Human Intelligence Task), a worker watches the video, then writes low-level, step-by-step *instructions* for each highlighted sub-goal segment. The worker also writes a high-level *goal* that summarizes what the robot should accomplish during the expert demonstration.

These directives are validated through a second HIT by at least two annotators, with a possible third tie-breaker. For validation, we show a worker all three language directive

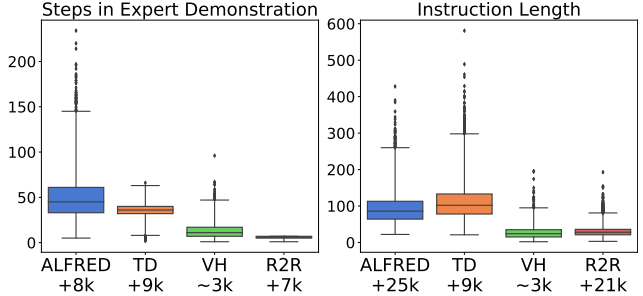


Figure 3: **Comparison to Existing Datasets.** Expert demonstration steps and instruction tokens of ALFRED compared to other datasets with human language for action sequences: Touchdown (TD) [14], VirtualHome (VH) [42], and Room-to-Room (R2R) [3]. The total number of demonstrations or annotations is given with the dataset label.

annotations without the video. The worker selects whether the three directives describe the same actions, and if not, which is most different. If a directive is chosen as most different by a majority of validation workers, it is removed and the demonstration is subsequently re-annotated by another worker. Qualitatively, these rejected annotations contain incorrect object referents (*e.g.*, “egg” instead of “potato”) or directions (*e.g.*, “go left towards...” instead of “right”).

4. Baseline Models

An agent trained for ALFRED tasks needs to jointly reason over vision and language input and produce a sequence of low-level actions to interact with the environment.

4.1. Sequence-to-Sequence Models

We model the interactive agent with a CNN-LSTM sequence-to-sequence (SEQ2SEQ) architecture. A CNN encodes the visual input, a bidirectional-LSTM generates a representation of the language input, and a decoder LSTM infers a sequence of low-level actions while attending over the encoded language. See Figure 4 for an overview and the supplementary material for implementation details.

Supervision. We train all models using imitation learning on expert trajectories. This ensures the language directives match the visual inputs. At each timestep, the model is trained to produce the expert action and associated interaction mask for manipulation actions.

We note that a DAgger-style [44] student-forcing paradigm in ALFRED is non-trivial, even disregarding language alignment. Obtaining expert demonstration actions on the fly in navigation-only datasets like R2R [3] only requires rerunning A^* . In ALFRED, on the fly demonstrations requires re-planning. In some cases re-planning is not

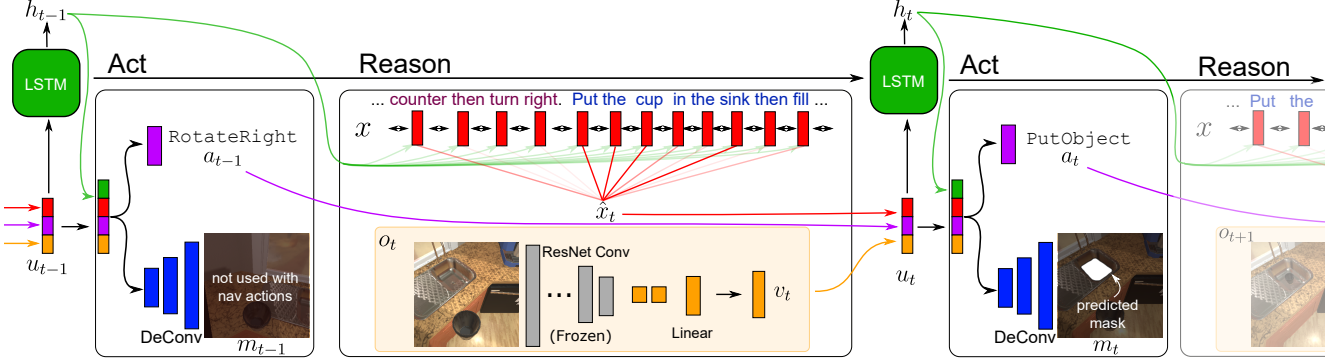


Figure 4: **Model overview.** At each step, our model reweights the instruction based on the history ($\hat{x}_t$), and combines the current observation features (v_t) and the previously executed action (a_{t-1}). These are passed as input to an LSTM cell to produce the current hidden state. Finally, the new hidden state (h_t) is combined with the previous features to predict both the next action (a_t) and a pixelwise interaction mask over the observed image to indicate an object.

possible: if during a task of **{Clean & Place, apple, refrigerator, KITCHEN-3}** a student-forcing model slices the only *apple* in the scene, the action cannot be recovered from and the task cannot be completed.

Visual encoding. Each visual observation o_t is encoded with a frozen ResNet-18 [22] CNN, where we take the output of the final convolution layer to preserve spatial information necessary for grounding specific objects in the visual frame. We embed this output using two more 1×1 convolution layers and a fully-connected layer. During training, a set of T observations from the expert demonstration is encoded as $\bar{V} = \langle v_1, v_2, \dots, v_T \rangle$, where v_t is the visual feature vector at time-step t .

Language encoding. Given a natural language goal $\bar{G} = \langle g_1, g_2, \dots, g_{L_g} \rangle$ of L_g words, and step-by-step instructions $\bar{S} = \langle s_1, s_2, \dots, s_{L_s} \rangle$ of L_s words, we append them into a single input sequence $\bar{X} = \langle g_1, g_2, \dots, g_{L_g}, \langle \text{SEP} \rangle, s_1, s_2, \dots, s_{L_s} \rangle$ with the $\langle \text{SEP} \rangle$ token indicating the separation between the high-level goal and low-level instructions. This sequence is fed into a bi-directional LSTM encoder to produce an encoding $x = \{x_1, x_2, \dots, x_{L_g+L_s}\}$ for each word in $\bar{X}$.

Attention over language. The agent’s action at each timestep is based on an attention mechanism weighting tokens in the instruction. We perform soft-attention on the language features x to compute the attention distribution α_t conditioned on the hidden state of the decoder h_{t-1} from the last timestep:

$$\begin{aligned} z_t &= (W_x h_{t-1})^\top x, \\ \alpha_t &= \text{Softmax}(z_t), \\ \hat{x}_t &= \alpha_t^\top x \end{aligned} \quad (1)$$

where W_x are learnable parameters of a fully-connected layer, z_t is a vector of scalar values that represent the attention mass for each word in x , and $\hat{x}_t$ is the weighted sum of x over the attention distribution α_t induced from z_t .

Action decoding. At each timestep t , upon receiving a new observation image o_t , the LSTM decoder takes in the visual feature v_t , language feature $\hat{x}_t$, and the previous action a_{t-1} , and outputs a new hidden state h_t :

$$\begin{aligned} u_t &= [v_t; \hat{x}_t; a_{t-1}], \\ h_t &= \text{LSTM}(u_t, h_{t-1}) \end{aligned} \quad (2)$$

where $[\cdot]$ denotes concatenation. The hidden state h_t is used to obtain the attention weighted language feature $\hat{x}_{t+1}$.

Action and mask prediction. The agent interacts with the environment by choosing an action and producing a pixelwise binary mask indicating a specific object in the frame. Although AI2-THOR supports continuous control for agent navigation and object manipulation, we discretize the action space. The agent chooses from among 13 actions. There are 5 navigation actions: MoveAhead, RotateRight, RotateLeft, LookUp, and LookDown together with 7 interaction actions: Pickup, Put, Open, Close, ToggleOn, ToggleOff, and Slice. Interaction actions require a pixelwise mask to denote the object of interest.¹ Finally, the agent predicts a Stop action to end the episode. We concatenate the hidden state h_t with the input features u_t and train two separate networks to predict the next action

¹The final object chosen by the interaction API is based on the Intersection-over-Union (IoU) score between the predicted mask and the ground-truth object mask from the simulator.

a_t and interaction mask m_t :

$$\begin{aligned} a_t &= \operatorname{argmax} (W_a [h_t; u_t]), \\ m_t &= \sigma (\text{deconv} [h_t; u_t]) \end{aligned} \quad (3)$$

where W_a are learnable parameters of a fully connected layer, **deconv** is a three-layer deconvolution network, and σ is a sigmoid activation function. Action selection is trained using softmax cross entropy with the expert action. The interaction masks are learned end-to-end in a supervised manner based on ground-truth object segmentations using binary cross-entropy loss. The mask loss is rebalanced to account for sparsity in these dense masks in which target objects can take up a small portion of the visual frame.

4.2. Progress Monitors

ALFRED tasks require reasoning over long sequences of images and instruction words. We propose two auxiliary losses (Eq. 4 & 5) that use additional temporal information to reduce this burden and form a sequence-to-sequence model with progress monitoring (SEQ2SEQ+PM).

Ma *et al.* [28] showed that agents benefit from maintaining an internal estimate of their progress towards the goal for navigation tasks. Akin to learning a value function in reinforcement learning, progress monitoring helps to learn the utility of each state in the process of achieving the overall task. Intuitively, this allows our agent to better distinguish between visually similar states such as just before putting an object in the microwave versus just after taking the object out. We introduce a simple module that predicts progress, $p_t \in [0, 1]$, conditioned on the decoder hidden state h_t and the concatenated input u_t :

$$p_t = \sigma (W_p [h_t; u_t]). \quad (4)$$

The supervision for p_t is based on normalized time-step values t/T , where t is the current time-step, and T is the total length of the expert demonstration (trained via L2 loss).

We also train the agent to predict the number of sub-goals completed so far, c_t . These sub-goals represent segments in the demonstration corresponding to sequences of actions like navigation, pickup, and heating as identified in the PDDL plan, discussed in Section 3.2. Each segment has a corresponding language instruction, but the alignment must be learned. This sub-goal prediction encourages the agent to coarsely track its progress through the language directive. This prediction is also conditioned on the decoder hidden state h_t and the concatenated input u_t :

$$c_t = \sigma (W_c [h_t; u_t]). \quad (5)$$

We train c_t in a supervised fashion by using the normalized number of sub-goals accomplished in the expert trajectory at each timestep, c_t/C , as the ground-truth label for a task with C sub-goals. We again train with an L2 loss.

5. Experiments

We evaluate the baseline models in the AI2-THOR simulator. When evaluating on test folds, we run models with the lowest validation loss. Episodes that exceed 1000 steps or cause more than 10 failed actions are terminated. Failed actions arise from bumping into walls or predicting action interaction masks for incompatible objects, such as attempting to `Pickup` a *counter top*. These limitations encourage efficiency and reliability. We assess the overall and partial success of models’ task executions across episodes.

5.1. Evaluation Metrics

ALFRED allows us to evaluate both full task and task goal-condition completion. In navigation-only tasks, one can only measure how far the agent is from the goal. In ALFRED, we can also evaluate whether task goal-conditions have been completed, for example that a *potato* has been sliced. For all of our experiments, we report both Task Success and Goal-Condition Success. Each Goal-Condition relies on multiple instructions, for example navigating to an object and then slicing it.

Task Success. Each expert demonstration is parameterized by a task to be performed, as illustrated in Figure 2. Task Success is defined as 1 if the object positions and state changes correspond correctly to the task goal-conditions at the end of the action sequence, and 0 otherwise. Consider the task: “Put a hot potato slice on the counter”. The agent succeeds if, at the end of the episode, any *potato slice* object has changed to the *heated* state and is resting on any *counter top* surface.

Goal-Condition Success. The goal-condition success of a model is the ratio of goal-conditions completed at the end of an episode to those necessary to have finished a task. For example, in the previous **Heat & Place** example, there are four goal-conditions. First, a *potato* must be sliced. Second, a *potato slice* should become *heated*. Third, a *potato slice* should come to rest on a *counter top*. Fourth, the same *potato slice* that is *heated* should be on the *counter top*. If the agent slices a *potato*, then moves a slice to the counter top without heating it, then the goal-condition success score is $2/4 = 50\%$. On average, tasks in ALFRED have 2.55 goal conditions. The final score is calculated as the average goal-condition success of each episode. Task success is 1 only if goal-condition success is 1.

Path Weighted Metrics. We include a Path Weighted version of both metrics that considers the length of the expert demonstration [2]. Expert demonstrations found via a PDDL solver on global information are not guaranteed to be optimal. However, they avoid exploration, use shortest path

Model	Validation				Test			
	Seen		Unseen		Seen		Unseen	
	Task	Goal-Cond	Task	Goal-Cond	Task	Goal-Cond	Task	Goal-Cond
NO LANGUAGE	0.0 (0.0)	5.9 (3.4)	0.0 (0.0)	6.5 (4.7)	0.2 (0.0)	5.0 (3.2)	0.2 (0.0)	6.6 (4.0)
NO VISION	0.0 (0.0)	5.7 (4.7)	0.0 (0.0)	6.8 (6.0)	0.0 (0.0)	3.9 (3.2)	0.2 (0.1)	6.6 (4.6)
GOAL-ONLY	0.1 (0.0)	6.5 (4.3)	0.0 (0.0)	6.8 (5.0)	0.1 (0.1)	5.0 (3.7)	0.2 (0.0)	6.9 (4.4)
INSTRUCTIONS-ONLY	2.3 (1.1)	9.4 (6.1)	0.0 (0.0)	7.0 (4.9)	2.7 (1.4)	8.2 (5.5)	0.5 (0.2)	7.2 (4.6)
SEQ2SEQ	2.4 (1.1)	9.4 (5.7)	0.1 (0.0)	6.8 (4.7)	2.1 (1.0)	7.4 (4.7)	0.5 (0.2)	7.1 (4.5)
+ PM PROGRESS-ONLY	2.1 (1.1)	8.7 (5.6)	0.0 (0.0)	6.9 (5.0)	3.0 (1.7)	8.0 (5.5)	0.3 (0.1)	7.3 (4.5)
+ PM SUBGOAL-ONLY	2.1 (1.2)	9.6 (5.5)	0.0 (0.0)	6.6 (4.6)	3.8 (1.7)	8.9 (5.6)	0.5 (0.2)	7.1 (4.5)
+ PM Both	3.7 (2.1)	10.0 (7.0)	0.0 (0.0)	6.9 (5.1)	4.0 (2.0)	9.4 (6.3)	0.4 (0.1)	7.0 (4.3)
HUMAN	-	-	-	-	-	-	91.0 (85.8)	94.5 (87.6)

Table 3: **Task and Goal-Condition Success.** For each metric, the corresponding path weighted metrics are given in parentheses. The highest values per fold and metric are shown in blue. All values are percentages.

navigation, and are generally efficient. The path weighted score p_s for metric s is given as

$$p_s = s \times \frac{L^*}{\max(L^*, \hat{L})} \quad (6)$$

where $\hat{L}$ is the number of actions the model took in the episode, and L^* is the number of actions in the expert demonstration. Intuitively, a model receives half-credit for taking twice as long as the expert to accomplish a task.

5.2. Sub-Goal Evaluation

Completing the entire sequence of actions required to finish a task is challenging. In addition to assessing full task success, we study the ability of a model to accomplish the next sub-goal conditioned on the preceding expert sequence. The agent is tested by first forcing it to follow the expert demonstration to maintain a history of states leading up to the sub-goal, then requiring it to complete the sub-goal conditioned on the entire language directive and current visual observation. For the task “Put a hot potato slice on the counter” for example, we can evaluate the sub-goal of navigating to the potato after using the expert demonstration to navigate to and pick up a *knife*. The tasks in ALFRED contain on average 7.5 such sub-goals (results in Table 4).

6. Analysis

Results from our experiments are presented in Table 3. We find that the initial model, without spatial or semantic maps, object segmentations, or explicit object-state tracking, performs poorly on ALFRED’s long-horizon tasks with high-dimensional state-spaces. The SEQ2SEQ model achieves $\sim 8\%$ goal-condition success rate, showing that the agent does learn to partially complete some tasks. This headroom (as compared with humans) motivates further research into models that can perform the complex vision-and-language planning introduced by ALFRED. The performance starkly contrasts other vision-and-

language datasets focused on navigation, where sequence-to-sequence with progress monitoring performs well [28].

6.1. Random Agent

A random agent is commonly employed as a baseline in vision-and-language tasks. In ALFRED, an agent that chooses a uniform random action and generates a uniform random interaction mask at each timestep achieves 0% on all folds, even without an API failure limit.

6.2. Unimodal Ablations

Previous work established that learned agents without visual inputs, language inputs, or both performed better than random agents and were competitive with initial baselines for several navigation and question answering tasks [50]. These performance gaps were due to structural biases in the datasets or issues with model capacity. We evaluate these ablation baselines (NO LANGUAGE and NO VISION) to study vision and language bias in ALFRED.

The unimodal ablation performances in Table 3 indicate that both vision and language modalities are necessary to accomplish the tasks in ALFRED. The NO LANGUAGE model finishes some goal-conditions by interacting with familiar objects seen during training. The NO VISION model similarly finishes some goal-conditions by following low-level language instructions for navigation and memorizing interaction masks for common objects like *microwaves* that are centered in the visual frame.

6.3. Model Ablations

We additionally ablate the amount of language supervision available to the model, as language directives are given as both a high-level goal and step-by-step instructions. Providing only high-level, underspecified goal language (GOAL-ONLY) is insufficient to complete the tasks, but is enough to complete some goal-conditions. Using just low-level, step-by-step instructions (INSTRUCTIONS-ONLY

performs similarly to using both high- and low-levels. Thus, this simple model does not seem to exploit the goal instruction to plan out sub-goals for step-by-step execution.

The two progress monitoring signals are marginally helpful, increasing the success rate from $\sim 1\%$ to $\sim 2\%$. Progress monitoring leads to more efficient task completion, as indicated by the consistently higher path weighted scores. They may help avoid action repetition and with the prediction of the `Stop` action.

The agent takes more steps than the expert in all cases, as indicated by the lower path weighted scores. Sometimes, this is caused by failing to keep track of state-changes, for example heating up an *egg* in the *microwave* multiple times. Further, the models also do not generalize well to *unseen* scenes, due to the overall visual complexity in ALFRED arising from new scenes and novel object class instances.

6.4. Human evaluation

We obtained a human evaluation of 100 randomly sampled directives from the *unseen* test fold. The experiment involved 5 participants who completed 20 tasks each using a keyboard-and-mouse interface. Before the experiment, the participants were allowed to familiarize themselves with AI2-THOR. The action-space and task restrictions were identical to that of the baseline models. Overall, the participants obtained a high success rate of 91%, while taking slightly longer than the expert with 86% path-length weighted success rate. This indicates that the directives in ALFRED are well-aligned with the demonstrations.

6.5. Sub-Goal Performance

We also examine performance of the SEQ2SEQ model on individual sub-goals in ALFRED. For this experiment, we use the expert trajectory to move the agent through the episode up to the sub-task. Then, the agent begins inference based on the language directive and current visual frame.

Table 4 presents path-length weighted success scores for 8 sub-goals. **Goto** and **Pickup** sub-tasks with the SEQ2SEQ+PM model achieve $\sim 51\%$ and $\sim 32\%$, respectively, even in *seen* environments. Visual semantic navigation is considerably harder in *unseen* environments. Similarly, interaction masks for **Pickup** actions in *unseen* environments are worse due to unfamiliar scenes and object instances. Simple sub-goals like **Cool**, and **Heat** are achieved at a high success rate of $\sim 90\%$ because these tasks are mostly object-agnostic. For example, the agent becomes familiar with using microwaves to heat things regardless of the object in-hand, because microwaves have little visual diversity across kitchens. Overall, the sub-goal evaluations indicate that models that exploit modularity and hierarchy, or make use of pretrained object segmentation models, may make headway on full task sequences.

		Sub-Goal Ablations - Validation							
		<i>Goto</i>	<i>Pickup</i>	<i>Put</i>	<i>Cool</i>	<i>Heat</i>	<i>Clean</i>	<i>Slice</i>	<i>Toggle</i>
<i>Seen</i>	Model								Avg.
	No Lang	28	22	71	89	87	64	19	90
	S2S	49	32	80	87	85	82	23	97
<i>Unseen</i>	S2S + PM	51	32	81	88	85	81	25	100
	No Lang	17	9	31	75	86	13	8	4
	S2S	21	20	51	94	88	21	14	54
<i>Unseen</i>	S2S + PM	22	21	46	92	89	57	12	32
									46

Table 4: **Evaluations by path weighted sub-goal success.** All values are percentages. The highest values per fold and task are shown in **blue**. We note that the NO VISION model achieves less than 2% on all sub-goals. See supplemental material for more.

7. Conclusions

We introduced ALFRED , a benchmark for learning to map natural language instructions and egocentric vision to sequences of actions. ALFRED moves us closer to a community goal of language-driven robots capable of navigation and interaction. The environment dynamics and interaction mask predictions required in ALFRED narrow the gap between what is required of agents in simulation and robots operating in the real world [37].

We use ALFRED to evaluate a sequence-to-sequence model with progress monitoring, shown to be effective in other vision-and-language navigation tasks [28]. While this model is relatively competent at accomplishing some sub-goals (e.g. operating microwaves is similar across **Heat & Place** tasks), the overall task success rates are poor. The long horizon of ALFRED tasks poses a significant challenge with sub-problems including visual semantic navigation, object detection, referring expression grounding, and action grounding. These challenges may be approachable by models that exploit hierarchy [8, 27], modularity [4, 16], and structured reasoning and planning [6]. We are encouraged by the possibilities and challenges that the ALFRED benchmark introduces to the community.

Acknowledgements

Thanks to our UW colleagues for helpful feedback, to Eli VanderBilt and Eric Kolve for their help with AI2-THOR and leaderboard setup, and Victor Zhong for early modeling design. And finally, thanks to Ranjay Krishna for sharing the Mechanical Turk annotation interface. This research was supported in part by the ARO (ARO-W911NF-16-1-0121), the NSF (IIS1252835, IIS-1562364, NSF-NRI-1637479), and the Allen Institute for Artificial Intelligence.

References

- [1] Jean-Baptiste Alayrac, Piotr Bojanowski, Nishant Agrawal, Ivan Laptev, Josef Sivic, and Simon Lacoste-Julien. Unsupervised learning from narrated instruction videos. In *CVPR*, 2016. 3
- [2] Peter Anderson, Angel Chang, Devendra Singh Chaplot, Alexey Dosovitskiy, Saurabh Gupta, Vladlen Koltun, Jana Kosecka, Jitendra Malik, Roozbeh Mottaghi, Manolis Savva, et al. On evaluation of embodied navigation agents. *arXiv preprint arXiv:1807.06757*, 2018. 6
- [3] Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould, and Anton van den Hengel. Vision-and-Language Navigation: Interpreting visually-grounded navigation instructions in real environments. In *CVPR*, 2018. 1, 2, 4, 13
- [4] Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. Neural module networks. In *CVPR*, June 2016. 8
- [5] Yoav Artzi and Luke Zettlemoyer. Weakly supervised learning of semantic parsers for mapping instructions to actions. *TACL*, 2013. 2
- [6] Masataro Asai and Alex Fukunaga. Classical planning in deep latent space: Bridging the subsymbolic-symbolic boundary. In *AAAI*, 2018. 8
- [7] Michael Beetz, Ulrich Klank, Ingo Kresse, Alexis Maldonado, Lorenz Mösenlechner, Dejan Pangercic, Thomas Rühr, and Moritz Tenorth. Robotic roommates making pancakes. In *IEEE-RAS*, 2011. 3
- [8] Yonatan Bisk, Daniel Marcu, and William Wong. Towards a dataset for human computer communication via grounded language acquisition. In *AAAI Workshop on Symbiotic Cognitive Systems*, 2016. 8
- [9] Yonatan Bisk, Deniz Yuret, and Daniel Marcu. Natural language communication with robots. In *NAACL*, 2016. 2
- [10] Mario Bollini, Stefanie Tellex, Tyler Thompson, Nicholas Roy, and Daniela Rus. Interpreting and executing recipes with a cooking robot. In *ISER*, 2012. 3
- [11] Angel Chang, Angela Dai, Thomas Funkhouser, Maciej Halber, Matthias Niessner, Manolis Savva, Shuran Song, Andy Zeng, and Yinda Zhang. Matterport3d: Learning from RGB-D data in indoor environments. *3DV*, 2017. 1
- [12] Devendra Singh Chaplot, Kanthashree Mysore Sathyendra, Rama Kumar Pasumarthi, Dheeraj Rajagopal, and Ruslan Salakhutdinov. Gated-attention architectures for task-oriented language grounding. In *AAAI*, 2017. 1, 2
- [13] David L Chen and Raymond J Mooney. Learning to interpret natural language navigation instructions from observations. In *AAAI*, 2011. 2
- [14] Howard Chen, Alane Suhr, Dipendra Misra, Noah Snaveley, and Yoav Artzi. Touchdown: Natural language navigation and spatial reasoning in visual street environments. In *CVPR*, 2019. 1, 2, 4
- [15] Abhishek Das, Samyak Datta, Georgia Gkioxari, Stefan Lee, Devi Parikh, and Dhruv Batra. Embodied Question Answering. In *CVPR*, 2018. 2
- [16] Abhishek Das, Georgia Gkioxari, Stefan Lee, Devi Parikh, and Dhruv Batra. Neural Modular Control for Embodied Question Answering. In *CoRL*, 2018. 8
- [17] Daniel Fried, Ronghang Hu, Volkan Cirik, Anna Rohrbach, Jacob Andreas, Louis-Philippe Morency, Taylor Berg-Kirkpatrick, Kate Saenko, Dan Klein, and Trevor Darrell. Speaker-follower models for vision-and-language navigation. In *NeurIPS*, 2018. 1, 2
- [18] Malik Ghallab, Adele Howe, Craig Knoblock, Drew McDermott, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. Pddl the planning domain definition language. 1998. 3
- [19] Daniel Gordon, Dieter Fox, and Ali Farhadi. What should i do now? marrying reinforcement learning and symbolic planning. *arXiv preprint arXiv:1901.01492*, 2018. 2
- [20] Daniel Gordon, Aniruddha Kembhavi, Mohammad Rastegari, Joseph Redmon, Dieter Fox, and Ali Farhadi. Iqa: Visual question answering in interactive environments. In *CVPR*, 2017. 2
- [21] Stevan Harnad. The symbol grounding problem. *Physica D*, 42:335–346, 1990. 1
- [22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 5, 12
- [23] Jörg Hoffmann and Bernhard Nebel. The ff planning system: Fast plan generation through heuristic search. *JAIR*, 2001. 3
- [24] Liyiming Ke, Xiujun Li, Yonatan Bisk, Ari Holtzman, Zhe Gan, Jingjing Liu, Jianfeng Gao, Yejin Choi, and Siddhartha Srinivasa. Tactical rewind: Self-correction via backtracking in vision-and-language navigation. In *CVPR*, 2019. 1, 2
- [25] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 13
- [26] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Daniel Gordon, Yuke Zhu, Abhinav Gupta, and Ali Farhadi. AI2-THOR: An Interactive 3D Environment for Visual AI. *arXiv preprint arXiv:1712.05474*, 2017. 1, 18
- [27] Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *NeurIPS*, pages 3675–3683, 2016. 8
- [28] Chih-Yao Ma, Jiasen Lu, Zuxuan Wu, Ghassan AlRegib, Zolt Kira, Richard Socher, and Caiming Xiong. Self-monitoring navigation agent via auxiliary progress estimation. In *ICLR*, 2019. 2, 6, 7, 8
- [29] Chih-Yao Ma, Zuxuan Wu, Ghassan AlRegib, Caiming Xiong, and Zolt Kira. The regretful agent: Heuristic-aided navigation through progress estimation. In *CVPR*, 2019. 1, 2
- [30] James MacGlashan, Monica Babes-Vroman, Marie desJardins, Michael L. Littman, Smaranda Muresan, Shawn Squire, Stefanie Tellex, Dilip Arumugam, and Lei Yang. Grounding english commands to reward functions. In *RSS*, 2015. 3
- [31] Matt MacMahon, Brian Stankiewicz, and Benjamin Kuipers. Walk the talk: Connecting language, knowledge, and action in route instructions. In *AAAI*, 2006. 2
- [32] Jonathan Malmaud, Earl Wagner, Nancy Chang, and Kevin Murphy. Cooking with semantics. In *ACL Workshop on Semantic Parsing*, 2014. 2

- [33] Dipendra Misra, Andrew Bennett, Valts Blukis, Eyvind Niklasson, Max Shatkhin, and Yoav Artzi. Mapping instructions to actions in 3D environments with visual goal prediction. In *EMNLP*, 2018. 2
- [34] Dipendra Misra, John Langford, and Yoav Artzi. Mapping instructions and visual observations to actions with reinforcement learning. In *EMNLP*, 2017. 2
- [35] Dipendra Misra, Jaeyong Sung, Kevin Lee, and Ashutosh Saxena. Tell me daved: Context-sensitive grounding of natural language to manipulation instructions. In *RSS*, 2014. 3
- [36] Dipendra Misra, Kejia Tao, Percy Liang, and Ashutosh Saxena. Environment-driven lexicon induction for high-level instructions. In *ACL*, 2015. 2
- [37] Arsalan Mousavian, Clemens Eppner, and Dieter Fox. 6-dof graspnet: Variational grasp generation for object manipulation. In *ICCV*, 2019. 1, 8
- [38] Khanh Nguyen and Hal Daumé III. Help, Anna! Visual Navigation with Natural Multimodal Assistance via Retrospective Curiosity-Encouraging Imitation Learning. In *EMNLP*, 2019. 2
- [39] Khanh Nguyen, Debadeepta Dey, Chris Brockett, and Bill Dolan. Vision-based navigation with language-based assistance via imitation learning with indirect intervention. In *CVPR*, 2019. 2
- [40] Daniel Nyga, Subhro Roy, Rohan Paul, Daehyung Park, Mihai Pomarlan, Michael Beetz, and Nicholas Roy. Grounding robot plans from natural language instructions with incomplete world knowledge. In *CoRL*, 2018. 3
- [41] Rohan Paul, Jacob Arkin, Derya Aksaray, Nicholas Roy, and Thomas M. Howard. Efficient grounding of abstract spatial concepts for natural language interaction with robot platforms. *IJRR*, 2018. 3
- [42] Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. Virtualhome: Simulating household activities via programs. In *CVPR*, 2018. 1, 2, 4
- [43] Michaela Regneri, Marcus Rohrbach, Dominikus Wetzel, Stefan Thater, Bernt Schiele, and Manfred Pinkal. Grounding action descriptions in videos. *TACL*, 2013. 2, 3
- [44] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011. 4
- [45] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A platform for embodied ai research. In *ICCV*, 2019. 1
- [46] Ozan Sener, Amir R. Zamir, Silvio Savarese, and Ashutosh Saxena. Unsupervised semantic parsing of video collections. In *ICCV*, 2015. 3
- [47] Mohit Shridhar and David Hsu. Interactive visual grounding of referring expressions for human-robot interaction. In *RSS*, 2018. 3
- [48] Hao Tan, Licheng Yu, and Mohit Bansal. Learning to navigate unseen environments: Back translation with environmental dropout. In *NAACL*, 2019. 1
- [49] Moritz Tenorth, Daniel Nyga, and Michael Beetz. Understanding and executing instructions for everyday manipulation tasks from the world wide web. In *ICRA*, 2010. 2
- [50] Jesse Thomason, Daniel Gordon, and Yonatan Bisk. Shifting the baseline: Single modality performance on visual navigation & qa. In *NAACL*, 2019. 7
- [51] Jesse Thomason, Michael Murray, Maya Cakmak, and Luke Zettlemoyer. Vision-and-dialog navigation. In *CoRL*, 2019. 2
- [52] Jesse Thomason, Shiqi Zhang, Raymond Mooney, and Peter Stone. Learning to interpret natural language commands through human-robot dialog. In *IJCAI*, 2015. 3
- [53] Xin Wang, Qiuyuan Huang, Asli Celikyilmaz, Jianfeng Gao, Dinghan Shen, Yuan-Fang Wang, William Yang Wang, and Lei Zhang. Reinforced cross-modal matching and self-supervised imitation learning for vision-language navigation. In *CVPR*, 2019. 1, 2
- [54] Xin Wang, Wenhan Xiong, Hongmin Wang, and William Yang Wang. Look before you leap: Bridging model-free and model-based reinforcement learning for planned-ahead vision-and-language navigation. In *ECCV*, 2018. 2
- [55] Erik Wijmans, Samyak Datta, Oleksandr Maksymets, Abhishek Das, Georgia Gkioxari, Stefan Lee, Irfan Essa, Devi Parikh, and Dhruv Batra. Embodied question answering in photorealistic environments with point cloud perception. In *CVPR*, 2019. 2
- [56] Haonan Yu and Jeffrey Mark Siskind. Grounded language learning from video described with sentences. In *ACL*, 2013. 3
- [57] Licheng Yu, Xinlei Chen, Georgia Gkioxari, Mohit Bansal, Tamara L. Berg, and Dhruv Batra. Multi-target embodied question answering. In *CVPR*, 2019. 2
- [58] Yuke Zhu, Daniel Gordon, Eric Kolve, Dieter Fox, Li Fei-Fei, Abhinav Gupta, Roozbeh Mottaghi, and Ali Farhadi. Visual semantic planning using deep successor representations. In *ICCV*, 2017. 2
- [59] Dimitri Zhukov, Jean-Baptiste Alayrac, Ramazan Gokberk Cinbis, David Fouhey, Ivan Laptev, and Josef Sivic. Cross-task weakly supervised learning from instructional videos. In *CVPR*, 2019. 3

Appendix A. Dataset Details

We give additional information about the generation of expert demonstrations in AI2-THOR, language directives, the annotation interface used to collect directives, and samples of annotations with their associated demonstrations.

A.1. Expert Demonstrations

When sampling task parameters, we employ an active strategy to maximize data heterogeneity. Figure F1 shows the distribution of high-level task across train, validation seen, and validation unseen folds. Figure F2 shows the distribution of subgoals across task types. And Figures F6 and F7 give the distributions of pickup objects and receptacles across the dataset. Each task parameter sample is defined by (t, s, o, r, m) , where

- t = the task type;
- s = the scene in AI2-THOR;
- o = the object class to be picked up;
- r = the final destination for o or $\emptyset$ for **Examine**;
- m = the secondary object class for **Stack & Place** tasks ($\emptyset$ for other task types).

To construct the next tuple, we first find the largest source of imbalance in the current set of tuples. For example if $o = \text{apple}$ is more common than $o = \text{plunger}$, $o = \text{plunger}$ will be ranked higher than $o = \text{apple}$. We additionally account for the prior distribution of each entity

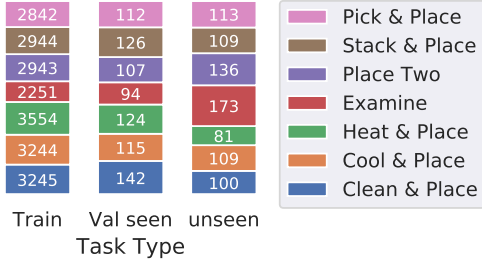


Figure F1: Task distribution across train, validation seen and unseen dataset splits.

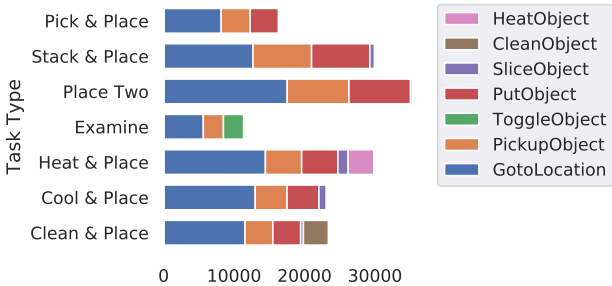


Figure F2: Subgoal distribution across 7 task types.

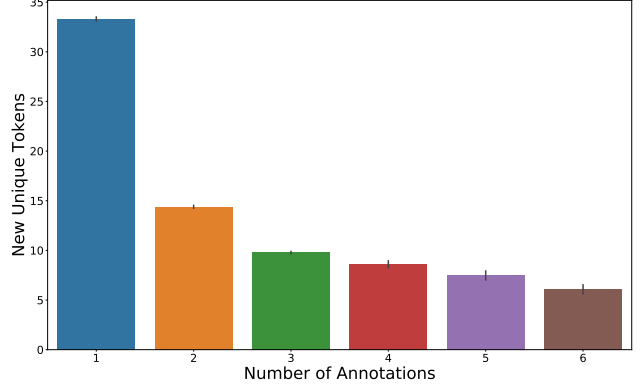


Figure F3: The number of unique tokens introduced per annotation of language directives.

(e.g., if *cup* is already represented in the data often as both o and m , it becomes disfavored by the sampling algorithm for all slots). We do this greedily across all slots until the tuple is complete. Given any partial piece of information about the task, the distributions of the remaining task parameters remain heterogeneous under this sampling, weakening baseline priors such as ignoring the language input and always executing a common task in the environment.

Once a task parameter sample is complete, the chosen scene is instantiated, objects and agent start position are randomized, and the relevant room data is encoded into PDDL rules for an expert demonstration. If the PDDL planner cannot generate an expert demonstration given the room configuration, or if the agent fails an action during execution, for example by running into walls or opening doors onto itself due to physical constraints, the episode is abandoned. We gather three distinct expert demonstrations per task parameter sample. These demonstrations are further vetted by rolling them forward using our wrapper to the AI2-THOR API to ensure that a “perfect” model can reproduce the demonstration. The full sampling generation and verification code will be published along with the dataset.

A.2. Example Language Directives

We chose to gather three directives per demonstration empirically. For a subset of over 700 demonstrations, we gathered up to 6 language directives from different annotators. We find that after three annotations, fewer than 10 unique tokens on average are introduced by additional annotators (Figure F3).

A.3. Annotation Interface

Figure F4 shows the Mechanical Turk interface used to gather language annotations. Workers were presented with a video of the expert demonstration with timeline segments indicating sub-goals. The workers annotated each segment

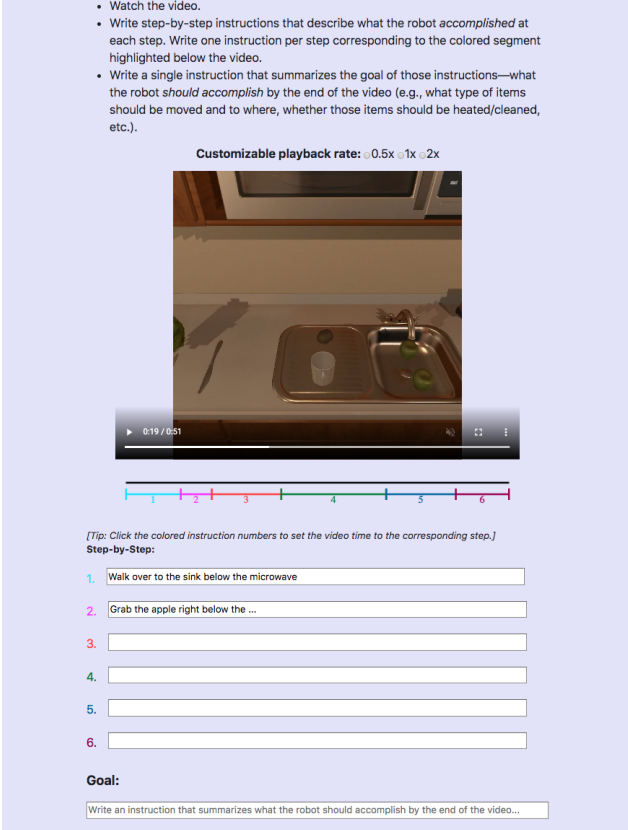


Figure F4: Mechanical Turk Annotation Interface.

while scrubbing through the video, and wrote a short summary description for the entire sequence. We paid workers \$0.7 per annotation. During vetting, annotators were paid \$0.35 per HIT (Human Interaction Task) to compare 5 sets of three directives each. These wages were set based on local minimum-wage rates and average completion time.

A.4. Vocabulary Distributions

Figure F8 shows vocabulary statistics of the language in ALFRED.

A.5. Dataset Examples

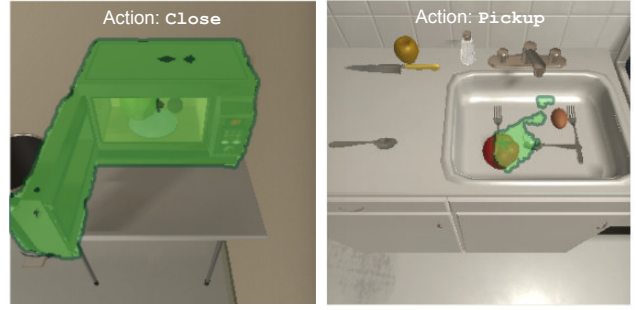
Figure F9 shows 7 expert trajectories (one per task type) and their accompanied annotations.

Appendix B. Implementation Details

We describe implementation and training details of our baseline Sequence-to-Sequence models.

Preprocessing We tokenize the language directives and convert all tokens to lower-case. During dataset generation, we save images from A12-THOR 300×300 pixels, and later resize them to 224×224 during training. The generation

Val Seen



Val Unseen



Figure F5: **Predicted interaction masks.** Masks generated by the SEQ2SEQ+PM model are displayed in green.

pipeline saves initialization information for objects and the agent, so all demonstration can be perfectly replayed in the simulator. Researchers can use this replay feature to augment the dataset by saving high-res images, depth maps, or object-segmentation masks.

Network Architecture We use a pretrained ResNet-18 [22] to extract $512 \times 7 \times 7$ features from the conv5 layer. These features are fed into a two-layer CNN with 1×1 convolutions to reduce the channel dimension from 512 to 64. The $64 \times 7 \times 7$ output is flattened, and a fully-connected layer produces a 2500-dimensional visual feature v_t .

The language encoder is a bi-directional LSTM with a hidden-dimension of 100. We do not use pretrained language models to initialize the LSTM, and the encodings are learned from scratch in an end-to-end manner. We also use a self-attention mechanism to attend over the encodings to initialize the hidden-state of the decoder LSTM.

The action decoder is an LSTM with a hidden-dimension of 512. The actor is a fully-connected layer that outputs logits for 13 actions. The mask decoder is a three-layer deconvolution network, which takes in the concatenated vector u_t and transforms it into $64 \times 7 \times 7$ features with a fully-connected layer. These features are subsequently up-scaled into a $1 \times 300 \times 300$ binary mask through three layers of deconvolutions and up-sampling with bi-linear interpolation.

Training The models were implemented with PyTorch and trained with the Adam optimizer [25] at a learning rate of $1e-4$. We use dropout of 0.3 on the visual features and the decoder hidden state, tuned on the validation data. Both the action and mask losses are weighted equally, while the auxiliary losses are scaled with a factor of 0.1. For evaluation, we choose models with the lowest loss on the validation *seen* set. It should be noted that, due to the nature of the tasks, low validation loss might not directly lead to better evaluation performance since the agent does not have to exactly imitate the expert to complete the task.

Notes on Random Agent Unlike discretized navigation where taking random actions might allow the agent to stumble upon the goal, ALFRED tasks are much harder to achieve by chance. The action space branching factor of Room-to-Room navigation [3], for example, is $4^6 \approx 4000$ (6 average steps and 4 navigation actions). By contrast, the ALFRED average branching factor is $12^{50} \approx 10^{53}$ (50 average steps for 12 actions). Beyond action type prediction, the ALFRED state space resulting from dynamic environments and the need to produce pixel-wise masks for interactive actions explodes further.

B.1. Predicted Masks

Figure F5 shows a few examples of masks generated by the SEQ2SEQ+PM model in seen and unseen validation scenes. The *Microwave* mask accurately captures the contours of the object since the model is familiar with receptacles in seen environments. In contrast, the *Sink* mask in the unseen scene poorly fits the unfamiliar object topology.

Appendix C. Additional Results

C.1. Performance by Task Type

In Table A1, we present success rates across the 7 task types. Even the best performing model, SEQ2SEQ+PM, mostly succeeds in solving some short-horizon tasks like **Pick & Place** and **Examine**. Long horizon tasks like **Stack & Place** and **Pick Two & Place** have near zero success rates across all models.

Task Type	Task Ablations - Validation					
	NO LANGUAGE		SEQ2SEQ		SEQ2SEQ+PM	
	<i>Seen</i>	<i>Unseen</i>	<i>Seen</i>	<i>Unseen</i>	<i>Seen</i>	<i>Unseen</i>
Pick & Place	0.0	0.0	6.3	1.0	7.0	0.0
Stack & Place	0.0	0.0	0.0	0.0	0.9	0.0
Pick Two	0.0	0.0	1.6	0.0	0.8	0.0
Clean & Place	0.0	0.0	0.0	0.0	1.8	0.0
Heat & Place	0.0	0.0	1.9	0.0	1.9	0.0
Cool & Place	0.0	0.0	2.4	0.0	4.0	0.0
Examine	0.0	0.0	4.3	0.0	9.6	0.0

Table A1: **Success percentages across 7 task types.** The highest values are shown in **blue**.



Figure F6: Pickup distributions in the train, validation *seen* and *unseen* folds.

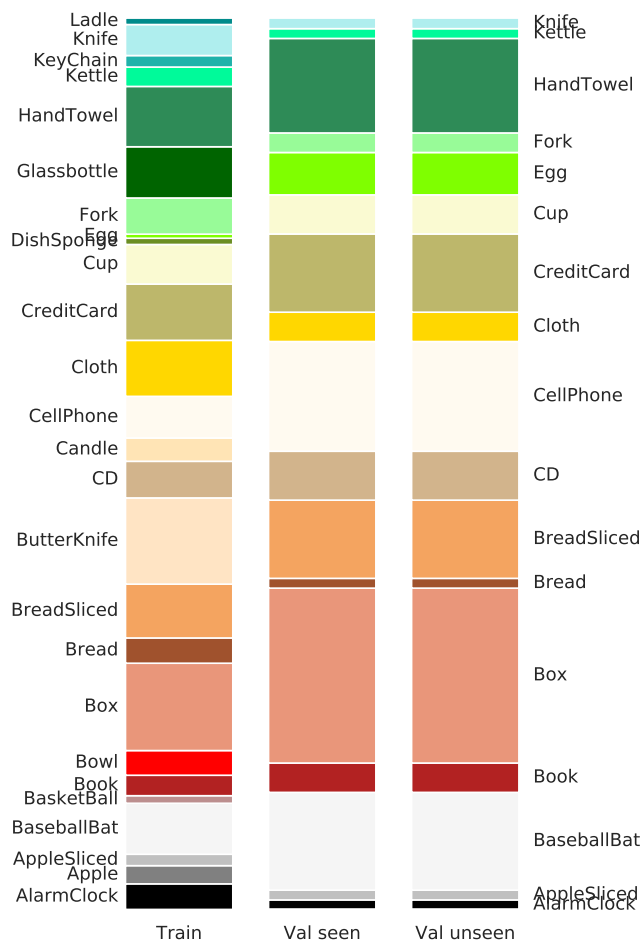


Figure F7: Receptacle distributions in the train, validation *seen* and *unseen* folds.

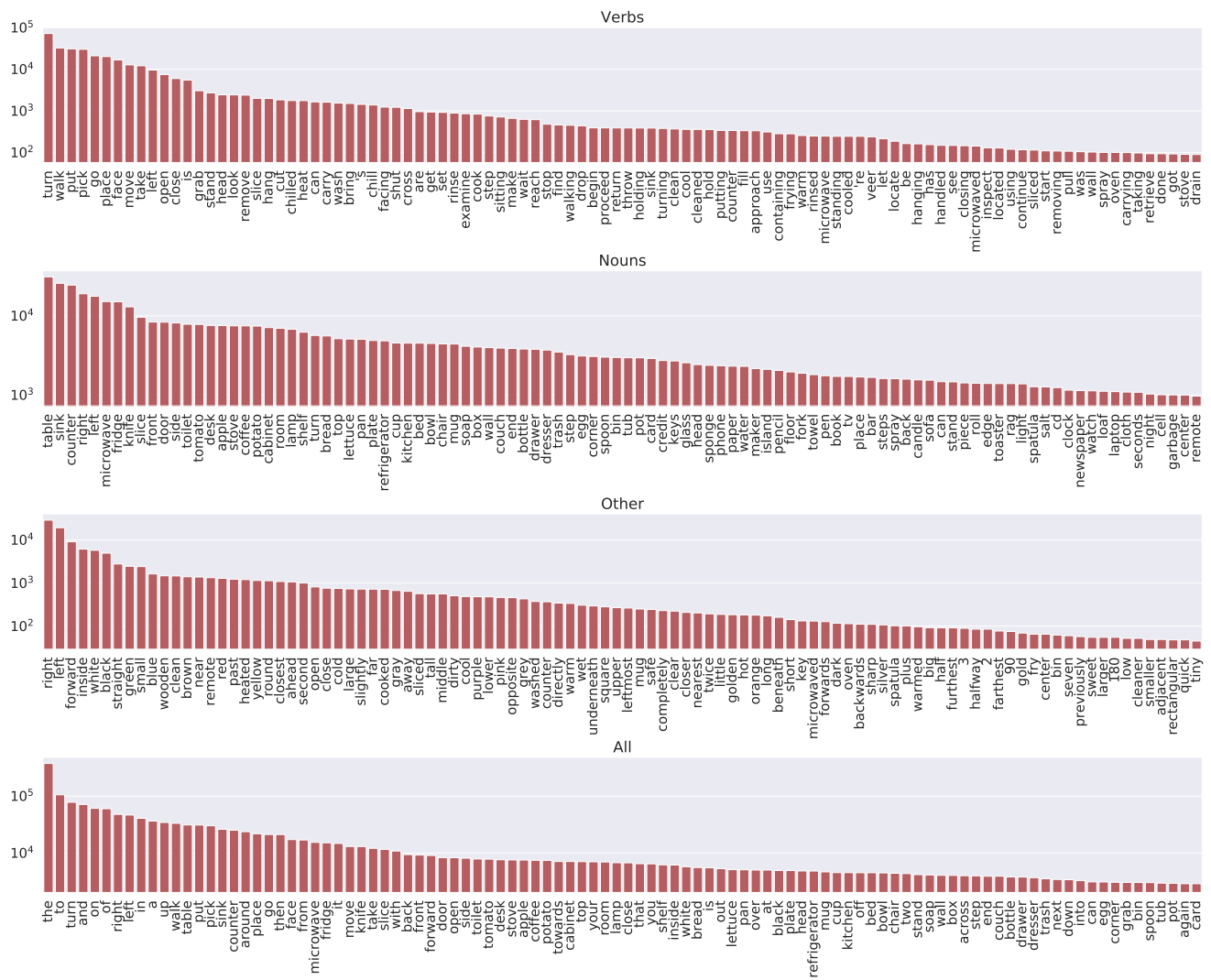


Figure F8: **Vocabulary Distributions.** Frequency distributions of 100 most common verbs, nouns, other words (non-verbs and non-nouns), and all words.

Pick & Place



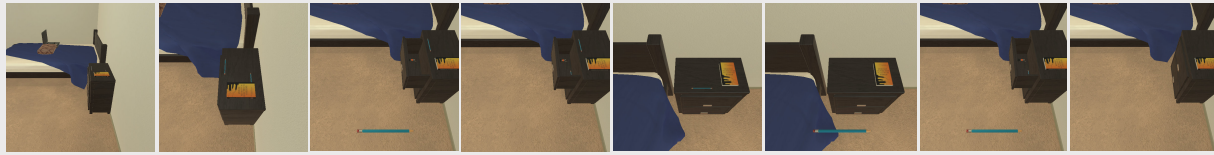
	Annotation # 1	Annotation # 2	Annotation # 3
Goals	Put a watch on a table.	Put the watch on the coffee table.	Move the watch to the coffee table.
Instructions	Go to the right and turn around to face the end of the cabinet with the television on it. Pick the watch up from the cabinet. Go to the right and then turn to face the coffee table in front of the couch. Put the watch on the table.	Turn right and go to the TV stand. Pick up the watch from the stand. Turn around and face the coffee table. Put the watch on the coffee table.	Turn right, go straight, turn right, step forward, then turn right to face the table with the TV on it. Pick up the watch on the table, to the right of the remote. Turn right, move forward, turn left, move forward, then turn right to face the coffee table. Put the watch on the front left corner of the coffee table.

Stack & Place



	Annotation # 1	Annotation # 2	Annotation # 3
Goals	Put a bowl with a spoon in it on the table.	Move a spoon and bowl to a table.	To put a spoon in a bowl plus moving them to the kitchen table.
Instructions	Turn left, and walk across the room to the microwave. Pick up the spoon. Turn left, and walk to the coffee machine. Put the spoon in the bowl. Pick up the bowl. Turn left, walk to the table, and turn left. Put the bowl on the table.	Move a spoon and bowl to a table. Go to the counter right of the microwave. Pick up the spoon from the counter. Go to the coffeemaker. Place the spoon in a bowl next to the coffeemaker. Pick up the bowl. Go to the black table. Place the bowl down on the table.	Turn left and walk across the room to face a spoon on the right end of the counter. Pick up the spoon on the end of the counter. Turn left and walk across the room to face the coffee maker on the counter. Put the spoon in the bowl to the right of the coffee maker on the counter. Pick up the bowl with a spoon in it on the counter. Turn left and walk across the room and turn left to face the kitchen table. Place the bowl with the spoon in it on the kitchen table.

Pick Two & Place



	Annotation # 1	Annotation # 2	Annotation # 3
Goals	Putting pencils inside of a cabinet.	Put two pencils in a drawer.	Place the two pencils in the stand.
Instructions	Walk to the bedside table in front of you. Grab the pencil that's on the table. Move slightly to the left and open the top cabinet of the bedside table. Place the pencil inside the cabinet. Face the front of the bedside table. Grab the pencil off of the bedside table. Open the top cabinet on the table. Place the pencil inside the cabinet and then close it.	Walk straight ahead to the end table. Pick up the blue pencil on the right. Walk around to the front of the end table. Put the pencil in the top drawer of the end table. Look up at the top of the end table. Pick up the pencil on the table. Look down at the drawer. Put the pencil in the drawer on top of the other pencil.	Walk to the stand next to the bed. Grab the pencil from the stand. Open the shelf inside of the stand. Place the pencil in the top shelf of the stand. Close the shelf, walk back to the stand. Grab the other pencil from the stand. Walk back to the stand next to the bed. Place the pencil in the top shelf of the stand.

Clean & Place



	Annotation # 1	Annotation # 2	Annotation # 3
Goals	Put a clean rag on the top shelf of a barred rack.	Wash the pink towel on the shelf, put it back on the shelf.	Clean a red cloth.
Instructions	Turn around, go to the barred rack. Pick up the rag from the bottle shelf of the barred rack. Go to the sink on the left. Put the rag in the sink, turn on then turn off the water. Go to the barred rack to the right of the sink. Put the rag on the top shelf of the barred rack.	Wash the pink towel on the shelf, put it back on the shelf. Turn around and go the shelf. Pick up the pink towel on the shelf. Turn around and put the towel in the sink. Fill the sink with water and wash the towel, take the towel out. Go back to the shelf. Put the towel back on the shelf.	walk on over to the towel drying rack, pick up a dirty red cloth from the towel rack, walk over to the left side of the bathroom sink, turn on the water to rinse the dirty red cloth and pick it back up again, walk back over to the towel drying rack, place the clean cloth on the drying rack.

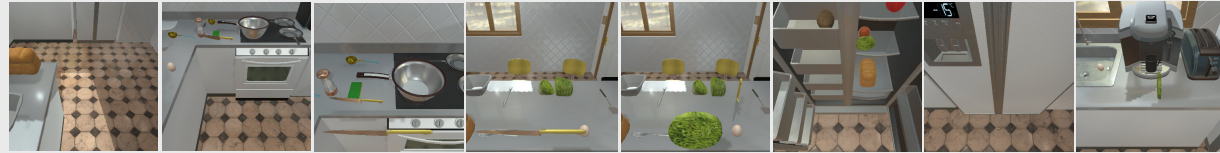
Figure F9: **Dataset Examples.** Annotations for seven expert demonstrations.

Heat & Place



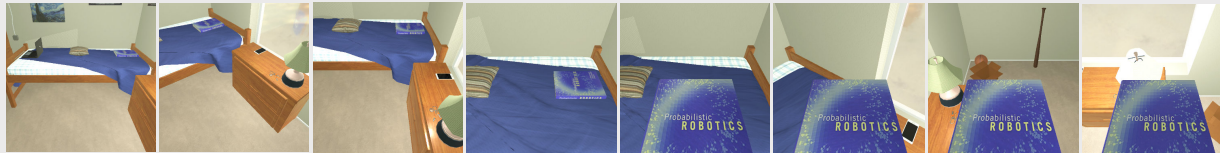
	Annotation # 1	Annotation # 2	Annotation # 3
Goals	Put a cooked potato slice on the counter.	Place a slice of cooked potato onto the counter.	Put a piece of cooked potato on the counter.
Instructions	Turn right, turn right, walk past the sink, turn left to face round table with tablecloth. Pick up the yellow-handled knife from the table. Cut a slice in the potato on the table. Turn left, turn left, turn right at counter, cross room, turn left at refrigerator to face counter. Put knife down on the table. Turn left, walk past sink, turn left to face round table. Pick the potato slice up from the table. Turn left, make right around corner of counter, turn left to face stove and microwave. Put potato in microwave, cook it, take it out of microwave. Turn right, cross room, turn left at counter with blue plate on it. Put potato on the counter in front of the blue plate.	Turn right, move to the table. Pick up the knife from the table. Slice the potato on the table. Turn left, move to the counter left of the bread. Put the knife on the counter near the soap container. Turn left, move to the table. Pick up a slice of potato from the table. Turn left, move to the counter in front of the stove. Put the potato slice into the microwave, cook it, pick it back up. Turn right, move to the counter left of the bread. Put the cooked potato slice on the counter.	Turn right and cross the room, then turn left and go to face the gray table. Pick up the knife from in between the lettuce and the apple. Use the knife to slice the potato that's on the gray table. Bring the knife with you and go face the kitchen counter with the loaf of bread. Put the knife down in front of the soap dispenser on the counter. Go back over to the gray table. Pick up a slice of the cut potato from the table. Bring the potato with you and go over to the stove, then look up at the microwave. Cook the potato slice in the microwave, then take it out again. Bring the potato slice over to the counter top with the loaf of bread and the knife you used to cut it. Put the potato slice down in front of the blue plate.

Cool & Place



	Annotation # 1	Annotation # 2	Annotation # 3
Goals	Put a slice of cold lettuce on a counter.	Put a chilled slice of lettuce on the counter.	Slice some lettuce and cool it in the refrigerator so you can put it on the counter top.
Instructions	Turn left, go forward past the counter, turn left, go forward to the counter to the left of the oven. Take the knife to the left of the large spoon from the counter. Turn around, go forward a step, turn right to the counter. Cut the lettuce on the counter into slices. Turn right, go forward a step, turn left to the counter. Put the knife behind the egg on the counter. Turn left, go forward a step, turn right to the counter. Take a slice of lettuce from the counter. Turn left, go forward, turn right to the fridge. Go to the fridge. Chill the lettuce in the fridge in front of the apple. Take the lettuce from the fridge. Turn left, go forward, turn right to face the coffee maker. Put the lettuce in front of the coffee maker on the counter.	Turn left, head toward the fridge, turn left and go to the stove. Pick up the knife beside the spoon on the counter. Turn around and turn to the right to face the counter with the egg. Cut the lettuce on the counter. Move right and back left to the counter. Put the knife behind the egg on the counter. Move the left and turn back right towards the counter. Pick up a slice of lettuce. Turn around and head to the fridge. Put the lettuce on the second shelf of the fridge, close the fridge, open the fridge and pick it back up. Turn left, go halfway across the room and turn right toward the coffee maker. Put the slice of lettuce on the counter in front of the coffee maker.	turn left and go around the counter top, then go straight to the stove top. pick up the knife with the yellow handle from behind the salt shaker on the counter top. turn left and face the counter top to your left. slice the lettuce on the counter top. face the counter top with the knife in hand. place the knife down next to the lettuce slices on the counter top. place the lettuce on the counter top. pick up a slice of lettuce on the counter top. turn left, then face the opposite wall behind you to face the refrigerator. open the refrigerator, and place the slice of lettuce in front of the apple one shelf above the bread, then shut the door and open it up again after several seconds to pick the lettuce slice up. turn left, then face forward to the part of the counter top on which the coffee maker sits. place the slice of lettuce in front of the coffee maker.

Examine in Light



	Annotation # 1	Annotation # 2	Annotation # 3
Goals	Read a book by lamp light.	Examine a book with a lamp.	Pick up a book and turn on a lamp.
Instructions	Head forward to the bed in front of you. Pick up the blue book that is sitting on the bed; the book that says Probabilistic Robotics. Turn to your right and walk to the night stand. Turn on the lamp that is sitting on the night stand.	walk forward a few steps, turn right, take two steps, turn left, walk to bed. pick up the book that is on the bed. turn around, take a step, turn left to face small table. turn the lamp on.	Walk forward to face the bed. Pick the book up from the bed. Turn to the right and face the night stand with the lamp. Turn the lamp on.

Figure F9: Dataset Examples. Annotations for seven expert demonstrations.



Figure F10: **Visual diversity of AI2-THOR [26] scenes.** Top to bottom rows: kitchens, living rooms, bedrooms, and bathrooms. Object locations are randomized based on placeable surface areas and class constraints. See <https://ai2thor.allenai.org/ithor/demo/> for an interactive demo.